Operating Systems

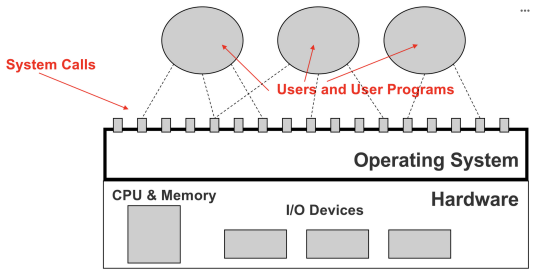
Michelangelo Prisciano

February 16, 2026

Contents

1 Week 1

1.1 What is an Operating System?



An operating system (OS) is software that manages the resources on the computer acting as an intermediary between the hardware and the user. If there wasn't one every program would need code to access hardware/resources. Operating systems have some core functions:

- Process Management: keeping different processes running smoothly at the same time without interfering with each other
- Memory Management: it ensures processes don't override each other's data
- File System Management: store, retrieve and manage data on the hard disk
- Device Management: ensures devices like mouse, keyboard, etc. work smoothly
- Security and Access Control: user authentication and data encryption

Device drivers are specialized software that allows the OS to communicate with and control hardware components, such as printers, graphics cards, etc.

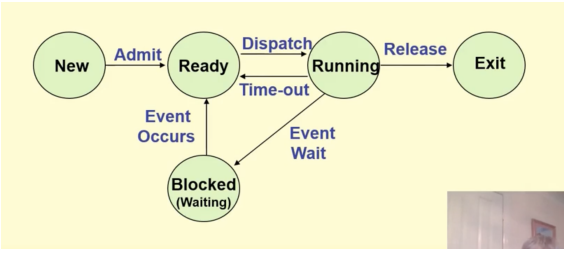
The *kernel* is the core component of an operating system and it operates at the lowest level, interacting directly with the hardware. Other higher-level software, on the other hand, interacts with the hardware via system calls. A kernel can follow two main design approaches: **monolithic** and **micro-kernel**. Each one has advantages/disadvantages but the main tradeoff is between performance and reliability/security.

In the **monolithic** approach, everything runs in kernel mode as one big program. This yields better performance as components can call each other directly and there's no overhead from context switching between kernel and user space. However, if there's a bug the entire kernel goes down (blue screen of death) and since everything has hardware access a vulnerability anywhere is a vulnerability everywhere. Whenever a new device needs to be added, the kernel needs to be recompiled with the new driver integrated into it.

The **micro-kernel** approach minimizes the kernel's responsibilities by stripping it down to only the most essential functions such as process management, memory management and basic inter-process communication. Other functionalities are delegated to user-space processes that run outside the kernel. This also means that there is a lot of context switch between kernel space and user space.

1.2 Processes

A process is a "program in execution", for example Firefox. A program by itself is not a process: a program becomes a process when an executable file is loaded into memory. As the process executes it goes through various states which can be described in the "Five-state process model" in the picture below:

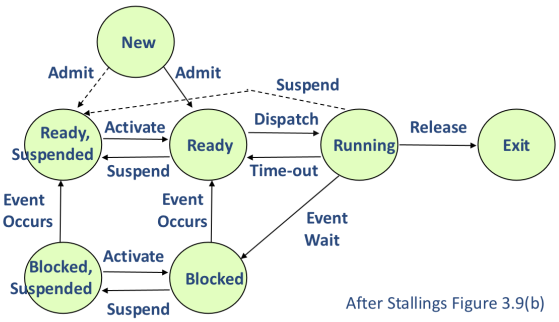


A new process is usually placed in the *ready* queue. It waits there until it is selected for execution, or it is *dispatched*. Once the process is allocated the CPU and is running, one of the following events could occur:

- The process could issue an I/O request and then be placed in an I/O queue
- The process could create a new subprocess and wait for the subprocess's termination
- The process could be removed forcibly from the CPU as a result of an interrupt, and be put back in the ready queue

What happens if the user decides to open a new process but the memory is already full? We resort to "virtual memory", that is auxiliary storage. However for processes to be in the running state, they need to be in the main memory. Therefore the five-state model needs to be updated to allow processes not in the main memory. We need to add a "suspend" state.

What happens if the process is unblocked but cannot be loaded into the main memory yet? We can add two suspended states.



A process image is all the data and code associated with the process. When this is loaded in main memory, this is made up of:

- User data
- User program code
- System stack
- Context
- Process Control Block (PCB). This is stored in kernel memory for each process.

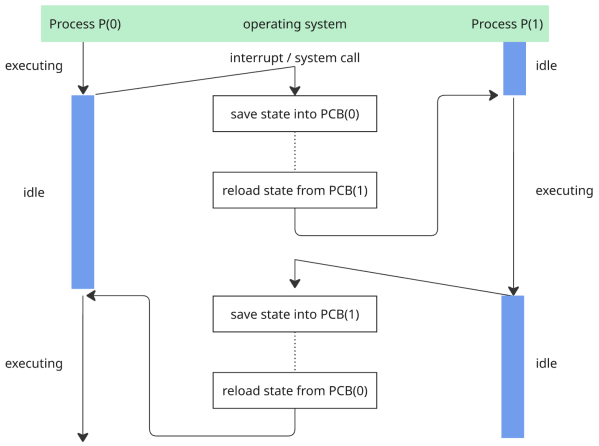
In particular, the PCB is how every process is represented in the operating system. This includes many pieces of information, including:

- Process state → new, ready, suspended, etc
- Program counter → address of the next instruction to execute
- CPU registers
- CPU-scheduling information → process priority, pointers to scheduling queues
- I/O status information → list of I/O devices allocated to the process, a list of open files, etc

Process Switching is the action of taking one process off the CPU and putting another one on. The OS forcibly interrupts a running process to give CPU time to do another process in three cases:

- External interrupt (for example, timers)
- Trap (exceptions or fault on processor)
- Supervisor/system call.

As mentioned above, context is information stored within the CPU registers about the state of the system. When we are doing a context switch we have to save all of that data and then reload it once we get back to the process to resume computation.



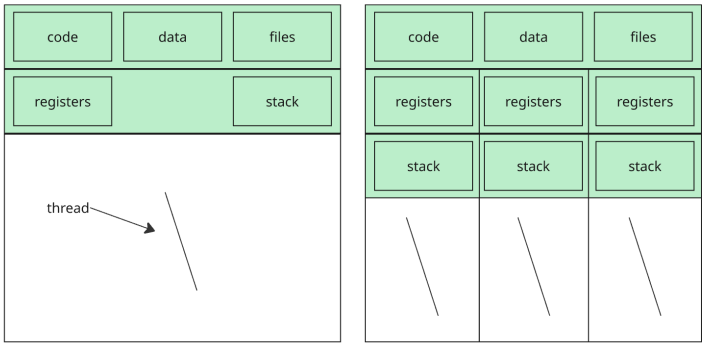
As processes enter the system, they are put into a job queue where the process **scheduler** selects a process from the Ready queue for program execution in the CPU. In particular, the long term scheduler (job scheduler) controls which processes are brought from disk into main memory, determining the degree of multiprogramming by managing how many processes can reside in RAM simultaneously. It executes infrequently—every few seconds or minutes—and aims to maintain a balanced mix of CPU-bound and I/O-bound processes to optimize overall system resource utilization. In contrast, the short-term scheduler (CPU scheduler) operates on processes already in memory, deciding which ready process should receive CPU time next. It must execute extremely quickly, typically every few milliseconds, since it runs very frequently and directly impacts system responsiveness. While the long-term scheduler affects the system's throughput and workload composition, the short-term scheduler determines the immediate allocation of the processor among competing processes.

There are several ways in which the OS can decide which process to run next.

- First-come, first served (FCFS)
- Shortest job next/first (SJN/SJF)
- Round robin (RR): each process gets a "slice" of CPU called quantum
- Priority scheduling: each process is assigned a priority and the OS runs the highest-priority process next.
- Multi-level queue scheduling: processes are divided in different queues based on characteristics. Each queue may have a different scheduling algorithm
- Multi-level feedback queue: similar to MLQS but processes can move between queues based on some factors, for example time spent in the queue
- Shortest remaining time (SRT): the OS runs the process with the shortest remaining time to completion
- Lottery scheduling: random
- Earliest deadline first (EDF): processes are scheduled based on their deadlines, with the earliest deadline getting the highest priority

1.3 Threads

A process can be broken down into "threads" which are processes-within-processes that share the same context as the main process. The picture below explains better how resources are shared within threads of the same process.



The benefits of multi-threaded programming can be broken down in four main categories:

- Responsiveness → this allows a program to continue running even if part of it it is blocked or is performing a long operation
- Resource sharing → threads share the memory and resources of the process to which they belong to by default
- Economy → context switch is lighter than process switch
- Scalability → the benefits of multi-threading can be increased in a multi-processor architecture, where threads may be running in parallel on different processors and increase parallelism

1.4 Questions

- What does it mean for a process to run run outside the kernel VS user-space? How is that determined?
- What does it mean "virtualization"? OS takes a physical resource and transforms it into a more general, powerful and easy-to-use virtual form of itself?
- The slides mention that this is the main memory if there was only one process. Isn't the "main memory" talked about here the RAM? Why is the kernel there? Does that mean "kernel processes"?

1.5 Notes

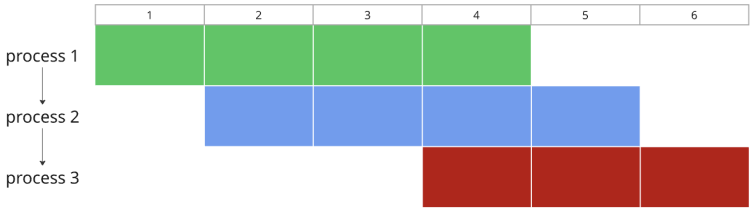
2 Week 2

2.1 Algorithms Analysis

This week we will go more in depth on the various scheduling algorithms.

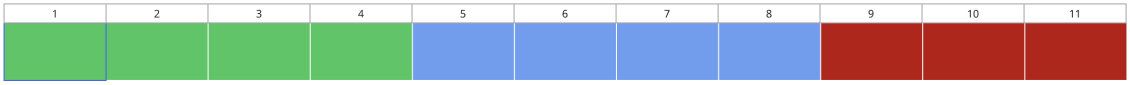
Let's define a few terms before proceeding:

- **burst/execution time** → the amount of CPU time a process needs to finish its current CPU work unit or job
- **arrival time** → the time when the job becomes ready/arrives in the system
- **completion time** → is the clock time when the job finishes
- **turnaround time** → completion time - arrival time
- **waiting time** → turnaround time - burst time
- **response time** → time from arrival until the job is first scheduled on the CPU (first run)



2.2 FIFO

Process 1 runs from 0 to 4 (excluded), Process 2 runs from 4 to 8 (excluded) and Process 3 runs from 8 to 11 (excluded).



Process	Completion Time	Turnaround Time	Waiting Time
Process 1	4	4 - 0 = 4	0
Process 2	8	8 - 1 = 7	7
Process 3	11	11 - 3 = 8	3

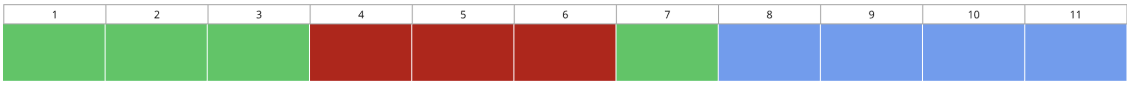
Table 1: FIFO Scheduling Performance Metrics

2.3 STCF

The Shortest time-to-completion First algorithm always runs the job with the smallest remaining execution time. If a new job arrives it can preempt the current job.

2.4 SJF

Process 1 will run from 0 to 3, then Process 3 will start and complete, then Process 1 will resume for the last CPU unit computation and then Process 2 will execute next.



Process	Completion Time	Turnaround Time	Waiting Time
Process 1	3 + 3 + 1 = 7	7 - 0 = 7	0
Process 2	3 + 3 + 1 + 4 = 11	11 - 1 = 10	6
Process 3	3 + 3 = 6	6 - 3 = 3	0

Table 2: Shortest Job First (SJF) Scheduling Performance Metrics

2.5 Round Robin

With the RR (Round Robin) algorithm we need to choose a proper size for the quantum. If it's too short there will be a lot of context switching, if it's too long then the algorithm will behave like FIFO.

Preemption definition: the OS interrupting (pausing) a running task and switching the CPU to another task with the intention to resume the first later.

2.6 Starvation Mitigating Techniques

In preemptive scheduling systems, starvation occurs when low-priority processes are perpetually delayed by higher-priority ones. Several strategies can mitigate this problem.

Aging involves gradually increasing the priority or effective weight of processes as they wait, ensuring that even low-priority jobs eventually become competitive for CPU time.

Hybrid policies combine priority-based and time-slice guarantees, allowing long-running jobs to execute for a guaranteed minimum duration before they can be preempted, which prevents complete starvation while maintaining responsiveness.

Another approach is to **limit preemption depth** by only allowing preemption when the remaining execution time difference between processes is substantial, which reduces excessive context switching and gives currently running processes a fair opportunity to make progress. These techniques balance system responsiveness with fairness, ensuring that all processes eventually receive CPU time.

2.7 CFS (Completely Fair Scheduler)

The Linux scheduler is called CFS (Completely Fair Scheduler) and it aims to allocate CPU time fairly across tasks using a *virtual runtime* (vruntime) metric. Assuming Task A and B have weight = 1 and Task C weight = 2, when a task runs for some real time its vruntime increases at different rates based on weight:

- Task A runs for 10ms → vruntime increases by 10
- Task B runs for 10ms → vruntime increases by 10
- Task C runs for 10ms → vruntime increases by 5 (half as much, because weight is 2x)
- Task C runs for 10ms → vruntime increases by 5 (half as much, because weight is 2x)

Since the scheduler picks up the process with the lowest vruntime, after 40ms of real time Task A and B got 10ms of CPU each (25%) while Task C got 20ms of CPU (50%).

2.8 MLQ and MLFQ

With the MLQ (Multi-Level Queue) approach processes are permanently assigned to a queue based on their type. On the other hand MLFQ (Multi-Level Feedback Queue) allows for processes to vary their priority based on their behaviour.

Processes start at the highest priority queue. CPU-intensive tasks that use their full time slice are demoted to lower queues while interactive processes that frequently perform I/O remain at high priority for responsiveness. To prevent gaming and starvation, MLFQ periodically boosts all processes back to the highest priority and tracks accumulated CPU usage at each level. This approach allows the scheduler to approximate Shortest Job First while maintaining excellent response times for interactive tasks.

2.9 Priority Inversion

Priority Inversion is an edge case that happens when a **high priority task is blocked waiting for a resource held by a low priority task**, while a medium priority task preempts the low priority one causing the high priority task to wait for the medium priority task.

- Task L starts running and acquires a shared resource (lock/mutex)

- Task H becomes ready and preempts L, trying to access the same resource that L is holding
- Task H blocks (goes into a waiting state) because it can't get the resource
- Since H is now blocked (not runnable), the scheduler looks for the next highest-priority runnable task
- Task M becomes ready to run and preempts L - H is blocked/waiting, M has higher priority than L so M runs instead of letting L continue
- L can't finish and release the resource because M keeps running
- H remains blocked waiting for the resource that L holds

The result is that H is indirectly waiting for M to finish, even though H > M in priority. There's a few solutions that can be put in place to avoid this:

- **Priority inheritance** → when a high priority task H blocks waiting for a resource held by low priority task L, the system temporarily boosts L's priority to match H's priority. This ensures that L can't be preempted by medium priority tasks
- **Priority ceiling** → each resource is assigned a "ceiling" priority - the highest priority of any task that might lock it. When a task acquires a resource, it temporarily runs at the ceiling priority
- **Reduce critical section length** → design to avoid holding locks when preemptible

2.10 Jain's Index

Jain's Index is a quantitative fairness metric to compare schedulers. It measures equality of resource allocations among n processes with allocations $x_1, \ldots, x_n$.

$$J(x_1, \ldots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \cdot \sum_{i=1}^{n} x_i^2}$$

- Range: $1/n$ (worst) to 1 (perfect fairness).
- If all processes equal, $J = 1$; if one process gets everything, $J = 1/n$.

2.11 Notes

3 Week 3

3.1 Concurrency

Concurrency is the ability of the operating system to execute multiple processes at the same time, potentially sharing resources. It's the backbone of modern computer systems which gives the illusion of multiple tasks being executed simultaneously also thanks to context switching techniques.

Common challenges in concurrency include sharing resources between multiple threads or processes. For example, if two processes try to modify the same data at the same time it can lead to unwanted consequences.

Some common issues arising from concurrency are:

- Race conditions → this means that the outcome of a program depends on the unpredictable timing of multiple threads accessing shared data
- Deadlock → a state where processes are stuck waiting on each other
- Starvation → some process uses a resource so much that no other processes can access it

Process can communicate with each other through inter process communications using mutex/semaphores or even more complex ways like messages (using message passing protocols, not usually provided by the OS)

3.2 Race Conditions

As mentioned before, this happens whenever two processes manipulate a common global resource in a way that yields an incorrect result due to the order of the operations.

```
local_copy = global_variable;
local_copy++;
global_variable = local_copy;
```

If two threads both execute the following piece of code we would expect the final global variable to be incremented by 2. However, if both threads access the variable at the same time they will both return the variable incremented by 1.

How do we fix concurrency issues then?

3.3 Mutual Exclusion

MutEx is a fundamental concept in concurrent programming that ensures only one thread or process can access a shared resource at any given time and by doing so it mitigates race conditions. Simply put, if one thread is accessing or modifying a shared resource other threads are temporarily blocked from accessing it until the first thread finishes.

Requirements of MutEx

- Must allows only one process/thread into a critical section of program code at any time
- A process halting in non-critical code should not interfere with any other process
- All processes/threads must be allowed into critical sections
- Critical sections are first-come, first-served
- Critical sections must have finite duration

We define a **critical section** as any piece of code that has to be executed under the assumption that no other process is accessing the same resources.

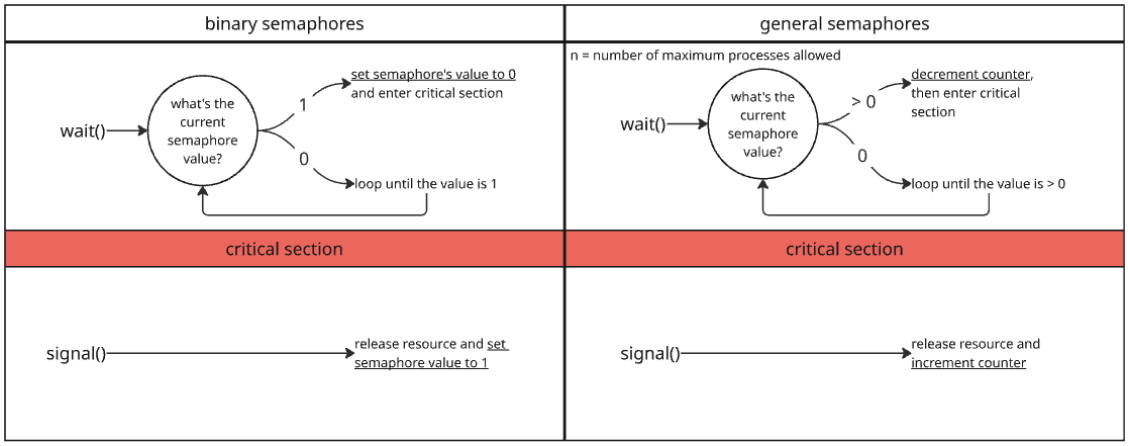
A simple implementation of MutEx at the OS level involves having a boolean variable in kernel protected space. If it's 1 the process must not proceed into critical section. If it's currently 0 then it's set to 1 and the process proceeds into the critical section. After it's been executed reset the boolean value to 0. This approach effectively stops the "interrupt service" (the one responsible for CPU preemptions) while executing a critical section.

Most modern CPUs have special machine code instructions to assist with mutual exclusion. A set of machine code instruction is an **atomic** action if it cannot be sub-divided or pre-empted.

3.4 Semaphores

A semaphore is a synchronization tool provided by the OS that allows two or more processes to communicate using signals during critical sections and it lives in a memory location shared by all processes that need to synchronize their operations.
A simple implementation is provided below.

```
do_something;
Wait(semaphore1); // Allocate a resource
  do_something_critical; // Use it for a while
Signal(semaphore1); // Return it to the pool
do_something else;
```

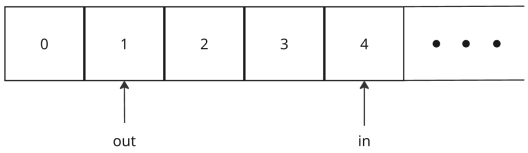


General semaphores contain two data structures: a FIFO queue of waiting processes and an integer counter initialized to the maximum number of concurrent processes allowed into the critical section. The counter is decremented every time a process raises the wait() syscall and incremented every time a process raises the signal() syscall.

Binary semaphores works in a very similar way but it only takes on the values of 0 and 1. The wait() syscall checks the semaphore value: if it's 0 the process is blocked, otherwise the value is changed from 1 to 0 and the process continues execution. The signal() operation sets back the semaphore value to 1.

While semaphores are simple, efficient and flexible for multiple uses they're also prone to errors since the programmer is responsible for ensuring the correct usage and it's difficult to prove programs are correct. Furthermore, the fact that processes have to co-operate when using semaphores breaks the isolation/modularisation of processes.

3.5 Producer/Consumer Problem



This is a classic problem where multiple producers concurrently generate and insert data into a **shared bounded buffer** (implemented as a circular FIFO queue), while a **single consumer** removes data from it. To maintain data integrity, only one agent (either a producer or the consumer) can access the buffer at any given time, which is enforced through mutual exclusion using **semaphores**. The semaphores coordinate access to the buffer, preventing race conditions and ensuring that producers don't overflow the buffer when it's full and the consumer doesn't attempt to retrieve data when it's empty.

The classic implementation involves three semaphores:

- bin = 1 → binary semaphore used to allow only one process in the critical section
- chars = 0 → general semaphore that tells us how many characters are in the buffer
- space = n → general semaphore that tells us how much space(s) is left in the buffer

The procedures involved are append() which appends an item to the buffer and increments the pointer **in** and take() which removes one item from the buffer and increments the pointer **out**.

Producer implementation

```
i = produce() // gets an item from somewhere
wait(space); // decrements space available on the buffer
wait(bin); // only one process allowed in the critical section
append(); // places item into the buffer
signal(bin); // lets another process into the buffer
signal(chars); // increment item count
```

Consumer implementation

```
wait(chars); // decrement item count
wait(bin); // only one process allowed in the critical section
i = take(); // removes item from buffer
signal(bin); // lets another process into the buffer
signal(space); // releases one space on the buffer
consume(i); // do something with the item
```

This approach heavily relies on the wait() and signal() functions being placed correctly around critical sections. It would be much easier and safer to declare which are the critical sections or shared resources and then let the OS ensure that those are accessed correctly: this exists and it's called **monitor**. It simplifies development by providing an automatic way to protect shared resources.

3.6 Message Passing

The minimum requirement for processes to talk and pass messages to each other is a pair of instructions:

- send(destination, message)
- receive(source, message)

3.7 Deadlock

Deadlock occurs when a group of processes or threads are stuck in a state where none of them can proceed because they are each waiting for resources that are held by the others in the group. This results in an infinite waiting loop where no process can release its resources or acquire the resources it needs to move forward. Deadlock is typically characterized by four conditions:

- Mutual exclusion: resources cannot be shared and only one process can use a resource at a time
- No preemption: resources cannot be forcibly taken from a process; the process must release them voluntarily
- Hold and wait: processes already holding a resource can request new ones
- Circular wait: there's a circular chain of processes where each one is waiting for a resource held by the next one in the chain

P_0	P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
.	.
.	.
.	.
`signal(S);`	`signal(Q);`
`signal(Q);`	`signal(S);`

In this example P0 executes wait(S) and P1 executes wait(Q). When P0 executes wait(Q) it must wait until P1 executes signal(Q), which will never happen because P1 is waiting for P0 to execute signal(S). Since these operations cannot be executed P0 and P1 are deadlocked.

3.8 Starvation

Starvation occurs when a process is perpetually denied access to the resources it needs because other processes continuously take priority. Unlike deadlock, where all processes are blocked, in starvation, some processes may be running and making progress while one or more processes are indefinitely delayed. Starvation typically happens when resource allocation is unfair, with some processes consistently being passed over in favor of others. For instance, in a system that uses priority-based scheduling, a low-priority process may never get CPU time if higher-priority processes keep arriving.

To prevent starvation, operating systems often implement fairness mechanisms, such as aging, where a process's priority increases the longer it waits. This ensures that even low-priority tasks eventually get the resources they need to execute.

3.9 Safe and unsafe states

A system is in a safe state if it can allocate resources to each process in such a way that all processes can complete without leading to deadlock. The state of resource allocation in the system is defined by the following data structures, where n is the number of processes and m is the number of resource types:

- **Available**: A vector of length m indicating the number of available resources of each type. If `Available[j]` $= k$, then k instances of resource type R_j are available.
- **Max**: An $n \times m$ matrix defining the maximum demand of each process. If `Max[i][j]` $= k$, then process P_i may request at most k instances of resource type R_j.
- **Allocation**: An $n \times m$ matrix defining the number of resources of each type currently allocated to each process. If `Allocation[i][j]` $= k$, then process P_i is currently allocated k instances of resource type R_j.
- **Need**: An $n \times m$ matrix indicating the remaining resource need of each process. If `Need[i][j]` $= k$, then process P_i may need k more instances of resource type R_j to complete its task. Note that `Need[i][j]` = `Max[i][j] - Allocation[i][j]`.

Consider a system with five processes P_0 through P_4 and three resource types A, B, and C with instances (10, 5, 7) respectively. Suppose at time t_0, we have the following snapshot:

	Allocation			Max			Need		
Process	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

Granted that the Total resources vector is provided, the **Available** vector is calculated as:

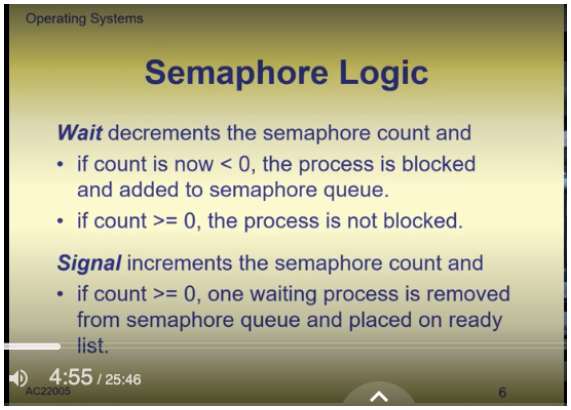
$$\text{Available} = \text{Total} - \sum_{i=0}^{4} \text{Allocation}[i] = (10, 5, 7) - (7, 2, 5) = (3, 3, 2)$$

1. Find a row in the Need matrix which is less than the Available vector. If such a row exists, then the process represented by that row may complete with those additional resources. If no such row exists, eventual deadlock is possible.
2. Looking ahead, pretend that that process has acquired all its needed resources, executed, terminated, and returned resources to the Available vector.
3. Repeat steps 1 and 2 until:
   (a) all the processes have successfully reached pretended termination; or
   (b) deadlock is reached (this implies the initial state was unsafe).

In addition to having advantages such as preventing starvation and deadlock by ensuring that the system remains in a safe state, the Banker's Algorithm also has several significant disadvantages that limit its practical application. It requires both the number of processes and resources to be fixed during execution, meaning no new processes can start and no resources can fail without risking deadlock. While the algorithm guarantees that all requests will be granted in finite time and that processes will eventually return their resources, this "finite time" could be impractically long—potentially leading to resource starvation where some processes wait indefinitely while others are prioritized. Additionally, all processes must declare their maximum resource requirements in advance, which is often unrealistic in dynamic systems where resource needs may not be known beforehand or may change during execution.

3.10 Questions

- No assumption about speed or quantity (number) of processes involved. Their speed should not have influence. Doesn't it though?
- When talking about semaphores we said that the process is allowed into the critical section if the counter is greater than 1. Shouldn't it be greater or equal to 1?
- How often are semaphores checked when a process is waiting?
- Confusing thing in the Signal section. It should be count greater than 0, not greater or equal

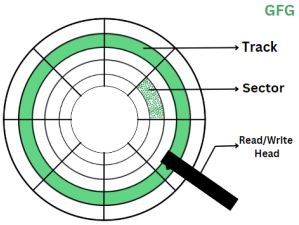


3.11 Notes

4 Week 4

4.1 HDDs and SSDs

A **hard disk** is a storage device used to store and retrieve digital data in computers. It consists of spinning disks (also called platters) coated with a magnetic material, and a read/write head that accesses data stored on the surface of these disks. The basic working principle of a hard disk relies on magnetic storage: bits of data are represented by the magnetization of tiny regions on the disk surface. To store a "1", the magnetic particles are aligned in one direction, and for a "0", they are aligned in the opposite direction. The read/write head, which floats just above the disk, changes the magnetization of these particles to store data and senses it to read data.



The hard disk is divided into **blocks** — the smallest unit of data that can be read or written to the disk. These blocks are further organized into **sectors**, which are typically 512 bytes or 4096 bytes in size. Sectors are grouped into **tracks** which are concentric circles on the surface of the disk platter: each track contains an array of magnetic cells. A collection of tracks at the same radius on all the platters in the disk is called a **cylinder**. In order to access the desired block of data from the hard disk the disk spins, the read/write head moves to the correct cylinder and finally to the appropriate track and sector.

In contrast to HDDs, solid-state drives (SSDs) use a completely different mechanism to store data. Instead of spinning disks and mechanical parts, SSDs rely on flash memory — electronic circuits that store data as electrical charges. SSDs have no moving parts, making them faster, more durable, and quieter than traditional hard drives. Data in SSDs is stored in pages and blocks, with the ability to access data almost instantly, regardless of its location in the storage. While SSDs tend to be more expensive than HDDs, their speed advantage makes them ideal for tasks that require fast data access, such as booting up operating systems or running high-performance applications.

4.2 Hard Disk I/O

The access time needed to reach a given block on a hard disk is made up of two values:

- Seek Time → time to move arm to the required track
- Rotational Latency → time for the correct block to come under the head

Similar to CPU scheduling, requests to access a certain block inside the hard disk will be put into a queue. **Disk scheduling** consists of deciding which requests should go next and choosing this in a smart way is the only way of reducing seek time and boost overall access and response time.

When we design a scheduling algorithm for a disk we need to keep in mind that the more local the data we need to access on the disk is the better the access time will be since there will be less mechanical motion between accessing the various blocks.

Two important OS concepts to remember:

- **Locality of Reference** → This is the tendency of a program to access a relatively small portion of its address space at any given time. Put simply they don't access memory randomly — they access data and instructions that are "close" to each other, either in time or in space.
- **Fragmentation of Data** → This happens when memory is used inefficiently and data gets scattered across the storage disk rather than being stored in one continuous block. This is a problem for two reasons: the disk has to physically move to multiple locations to read one file and if multiple processes are trying to access different scattered pieces of data they have to compete for disk access.

4.3 I/O Algorithms

Algorithm	Description	Advantages	Disadvantages
FIFO (First-In, First-Out)	I/O requests are handled in the order they arrive. It works well for a small number of processes.	Simple and fair, as all requests are treated equally.	Inefficient for HDDs because it ignores physical data locations, leading to high seek times. Less problematic for SSDs but still not optimal for wear leveling.
SSTF (Shortest Service Time First)	The request closest to the current position of the read/write head is serviced next.	Minimizes seek time for HDDs by servicing nearest requests first, improving performance.	Can lead to starvation of distant requests. SSDs benefit less since access time differences are minimal.
LOOK	The disk arm moves toward the nearest request in one direction to service all requests in that direction until the last one, then reverses direction. Unlike SCAN, it only goes as far as the last request in each direction.	Reduces unnecessary head movement in HDDs compared to SCAN, improving efficiency.	Since the algorithm moves in one direction at a time, if a new request arrives nearby the arm won't handle it immediately if it's moving in the opposite direction.
C-LOOK (Circular LOOK)	Like LOOK, but the arm only moves in one direction. When it reaches the last request, it jumps back to the start without servicing requests on the way back.	Same as LOOK.	Slightly reduces efficiency for HDDs due to jump-back, but improves fairness. For SSDs, this is mostly irrelevant.
LIFO (Last-In, First-Out)	The most recent I/O request is handled first.	Useful when newer data is more likely to be relevant or accessed again since it's likely to belong to nearby blocks.	Can starve older requests; impractical for HDDs. May cause uneven wear on SSDs.
Priority-Based Scheduling	Each I/O request is assigned a priority; higher-priority requests are serviced first.	Allows critical requests to be processed faster, improving performance for time-sensitive applications.	Lower-priority requests may be starved; reduces fairness and can lead to uneven SSD wear.

4.4 Advanced I/O Scheduling

In addition to the ones mentioned there's also other algorithms that try to optimize performance for specific types of applications.

- **Deadline Scheduler** → Used in database systems or any environment where it's important that requests are serviced before a given time.
- **CFQ (Completely Fair Queuing)** → Designed for systems where fairness between processes is important - for example, multi-user systems. I/O requests are grouped by process and each process gets a time slice during which the requests are serviced.
- **Anticipatory Scheduler** → Good for systems where block access can be easily predicted - for example, web servers.

While Linux allows to choose a certain disk I/O scheduler, Windows doesn't and allows only to use the default one.

4.5 RAID: Redundant Array of Independent Disks

RAID (Redundant Array of Independent Disks) is a data storage technology that combines multiple physical disks into a single logical unit to improve performance and provide fault tolerance. It uses a concept known as "striping": imagine you have a large file - let's say a 100MB video. With striping, instead of storing that entire file on one disk, RAID 0 breaks it into smaller chunks and distributes them across multiple disks. Chunk 1 (first 50MB) would go into Disk 1 and Chunk 2 (next 50MB) would go into Disk 2. In this way when the file is read both disks work simultaneously, each one reading their 50MB portion at the same time.

RAID	Description	Advantages	Disadvantages
RAID 0	Splits data evenly across two or more disks (striping). Boosts read and write performance since multiple disks can operate simultaneously.	Maximum performance improvement for both reads and writes, as all disks are utilized in parallel.	No redundancy—if one disk fails, all data is lost, making it unsuitable for critical data storage.
RAID 1	Stores identical copies of data on two or more disks, ensuring that if one disk fails, the other(s) can still provide the data.	High fault tolerance, as data is mirrored. If one disk fails, data remains available.	Doubles storage requirements, since every disk is an exact copy of the other. Write performance can be slower due to data being written to multiple disks.
RAID 2	Uses bit-level striping across multiple disks and adds error correction through Hamming codes. Rarely used in practice due to its complexity and the availability of more efficient RAID levels.	Fault tolerance with automatic error correction.	Requires log(N) additional disks for N main disks and is inefficient for modern hardware as most drives now include built-in error correction.
RAID 3	Stripes data at the byte level across multiple disks and uses a separate, dedicated disk to store parity information, which can be used to recover data in case of a disk failure.	Good read performance and fault tolerance.	The dedicated parity disk can become a bottleneck for write operations, and byte-level striping is inefficient for smaller I/O operations.
RAID 4	Similar to RAID 3, but stripes data at the block level rather than the byte level. A dedicated parity disk provides fault tolerance.	Efficient for larger I/O operations, with good read performance due to block-level striping.	The dedicated parity disk remains a potential bottleneck during write operations.
RAID 5	Stripes data at the block level and distributes parity information across all disks instead of storing it on a single dedicated disk.	Good balance of performance, redundancy, and storage efficiency. Fault tolerance is achieved without sacrificing too much storage space.	Write operations are slower due to the need to calculate and write parity information. If a disk fails, performance during the rebuild process can be significantly impacted.
RAID 6	Similar to RAID 5 but uses two sets of distributed parity blocks, providing extra redundancy. This allows the array to tolerate the failure of up to two disks simultaneously.	High fault tolerance, as it can handle two disk failures, making it more reliable than RAID 5 for large arrays.	Write performance is slower than RAID 5 due to the additional parity calculation. Also, more storage space is used for parity.

4.6 Filesystems

A filesystem is a high level interface for storage of organized data which abstracts the hardware details. Put simply, it presents the storage in a logical way for users with files and folders and keeps track of some information about the data (for example, which users can access the data, etc). Filesystems can also be used on other storage systems apart from HDDs and SSDs. All related data (cluster) should be stored in close by blocks on the disc in line with the locality of reference principle.

Talking about file allocation strategies, we have two options:

- **Pre-allocation** → all the space is allocated when file is created, even before anything is written to it. This requires either knowing in advance the file size or giving an estimated file size in advance. This which could be wrong and waste space if the requested size is too big compared to what we actually used.
- **Dynamic allocation** → space allocated when data is actually written. Files can grow and shrink therefore we don't need to give an estimation or waste space.

Filesystem	Description	Advantages	Disadvantages
FAT (File Allocation Table)	One of the oldest and simplest filesystems, developed in the late 1970s. It stores a table at the beginning of the disk mapping keeping track of portions allocated to a file. Versions include FAT12, FAT16, and FAT32, with FAT32 being the most common.	FAT32 is supported by nearly all operating systems, making it ideal for USB drives, SD cards, and removable media. It is simple, lightweight, and suitable for low-resource devices.	Limited to a maximum file size of 4 GB and a partition size of 2 TB. Lacks advanced features like journaling, making it less reliable for modern or high-performance systems.
NTFS (New Technology File System)	Developed by Microsoft and introduced with Windows NT. Includes modern features such as file permissions, encryption, compression, and journaling to ensure data integrity. Supports very large files and partitions.	Provides superior reliability and security through journaling and access control lists (ACLs). Ideal for modern Windows systems and large storage volumes. Supports large files and drives, making it scalable and future-proof.	Primarily designed for Windows, with limited support on macOS and Linux. Linux can read NTFS, but write support can be unreliable in some cases.
EXT4 (Fourth Extended Filesystem)	Default filesystem for most Linux distributions. Evolved from EXT2 and EXT3 with features like journaling, extents to improve file allocation, and delayed allocation for optimized writes.	Delivers excellent performance, scalability, and reliability. Journaling ensures data integrity, while extents and delayed allocation reduce fragmentation. Widely supported across Linux systems.	Primarily Linux-based, with limited support on Windows and macOS. Tools for non-Linux platforms exist but can be unreliable or difficult to configure.

We will look into these filesystems in more depth shortly.

4.6.1 FAT

The FAT filesystem is one of the oldest and most widely supported: its key feature is a table that keeps track of which memory blocks are associated to each file.

With **contiguous allocation** each file is stored in a number of contiguous blocks on the disk and each FAT table entry records the filename, the start block and the length. The DAT (Disc Allocation Table) keeps track of blocks or portions that are free on the disc as well as bad blocks (blocks where the disk failed to read/write). We **delete** a file on a FAT filesystem simply by removing the FAT entry associated with that file. We **add** a file by finding space in the DAT, removing that entry and then create a new entry in the FAT table.

Internal fragmentation can occur because files are stored in fixed-size units called clusters: each file must occupy a whole number of clusters even if it doesn't completely fill the last one and as a result the unused portion of that final cluster is wasted space — no other file can use it. For example, if the cluster size is 8 KB and a file is 10 KB, the file will need two clusters (16 KB total), leaving 6 KB unused in the second cluster. This wasted space adds up, especially when many small files are stored on a disk. The problem becomes worse as the cluster size increases since larger clusters mean more leftover unused space per file. This can be fixed with an operation called **de-fragmentation** which can be executed automatically by modern filesystems.

With **chained allocation** the FAT table contains the reference to the first block of each file and then each block contains data plus a reference to the next block. The last block will have a null reference to the next block. In this case there's less wasted space as well as slower random access. This method yields better performance for scenarios with predominantly sequential access to blocks.

With **indexed allocation** each FAT entry points to an **index block** instead of directly pointing to data blocks. Each index block, in turn, contains the full list of file portions for that file and it might look like this: (Block 5, Length 3), (Block 12, Length 5). This is the most popular file allocation method since it performs well for both random and sequential access.

Allocation Method	FAT Table Entries	Resolved Block Order
Contiguous (Normal)	FILE START LEN File1 5 4 File2 9 3 File3 12 6	File1: 5--8 File2: 9--11 File3: 12--17
Chained (Linked)	FILE START File1 5 File2 10 File3 15 *FAT (block → next)* 5 → 7, 7 → 9, 9 → −1 10 → 11, 11 → 13, 13 → −1 15 → 18, 18 → 19, 19 → −1	File1: 5 → 7 → 9 File2: 10 → 11 → 13 File3: 15 → 18 → 19
Indexed Allocation	FILE INDEX File1 8 File2 1 File3 5 START BLOCK LENGTH 10 2 14 3 16 5	File1: 10, 14, 16

As we mentioned already the DAT table is used to keep track of portions which are free in memory.

In the **Bit Table** approach we use an array with the same size of the number of blocks in the disc. Each entry can be either 0 if the block is free or 1 if it's used (or malfunctioning). Search for contiguous free blocks is a linear operation.

In the **Chaining Free Blocks** approach we treat the free space as a special file using the chaining method to link all free blocks together. In this case DAT is only a pointer to the first block in the chain of free blocks.

In the **Indexed Free Blocks** approach the treat the free space as a special file as well and DAT is simply the index block that we would store for a file under Indexed Allocation. This is the most efficient way of searching for free space. This is different

4.6.2 NTFS

NTFS (New Technology File System) was introduced in Windows NT and later versions. This was designed to handle volumes which can be a given disc, part of a disc or multiple discs. It includes advanced features such as file permissions, encryption, compression, and journaling. Each volume contains file system information, a collection of files and unallocated space (free space).

The **cluster** is the fundamental unit of disc space on NTFS and each one is made up of one or more consecutive sectors. The hierarchy is Disk - Sectors - Clusters - Files. Clusters don't need to be contiguous on the disc and the max file size is 2^{32} clusters.

In NTFS each file occupies entire clusters even if it doesn't completely fill the last one, meaning small files can waste some space. NTFS can work with disks that have different **sector sizes** — traditionally 512 bytes, but newer drives often use 4096-byte sectors. When handling large files or large disks, using a **larger cluster size** improves efficiency because the file system has fewer clusters to manage: think about the difference between handling 1GB on 4KB and 64KB clusters (262144 VS 16384 clusters). However, large clusters can lead to more wasted space if many small files are stored. The cluster size for a volume is **chosen during formatting**, when the filesystem is first created, and it remains fixed unless the drive is reformatted.

Internally the volume is divided into four sections:

- Partition Boot Sector: this has information on which filesystem is used and the boot loader.
- Master File Table: information about files stored on the volume and unallocated (free) space. This is similar to an "index block" except that it's stored in a relational database to search it more efficiently.
- System Files: this includes files that can help recover from unexpected system crashes or problems with the drivers (log files).
- Remaining Space: space used to store the data and free space.

4.6.3 EXT4

EXT4 (4th extended filesystem) is typically used in Linux. It supports very large discs and also very large files. It uses efficient allocation methods for both pre-allocation and dynamic allocation: for the latter one the actual allocation on disk space is done only when there is enough data to be written. Its journaling feature ensures data integrity, while extents and delayed allocation help reduce fragmentation and improve efficiency.

Clusters as a concept in NTFS are replaced by **extents**: these don't have a fixed size and they use ranges of contiguous blocks (i.e. first one and last one). Extents are dynamically created and stored into a tree data structure: at first the disk might have one big extent of free space, then when a new file is created that extent is **split** into an **allocated part** (for the file), and the **rest remains free**. When the file is deleted that extent becomes free and gets merged with other extents. If the file is too large it's split among several extents.

4.7 Week 4 Glossary

Portion	A contiguous chunk of space on disc made up of one or more consecutive blocks.
Contiguous data	Data stored in a sequence of clusters that are physically adjacent on the disk.
DAT (Disc Allocation Table)	Data structure that keeps track of blocks or portions that are free on the disc. It can also record blocks where the disc failed to read/write.

4.8 Week 4 Questions

- Difference between partition and volume?
- Sector, block, cluster. How many bytes in each one?
- Why cluster size is a power of 2?
- Partition Boot Sector: what is a boot loader?

4.9 Notes

5 Week 5

5.1 Security of Information

Nowadays technology has made it easy to collect, process, store, distribute and ultimately lose very large quantities of data. As we are more and more reliant on new technology, effective security continues to be of uttermost importance. Since computer systems are vulnerable security needs to constantly improve to keep up to date with external threats.

Operating systems play a crucial role in ensuring data security through several key mechanisms, including:

- **User Authentication:** The OS controls access to the system by requiring users to provide valid credentials (like passwords, biometric data, or tokens). This process ensures that only authorized users can log in and access resources.
- **Access Control:** Once a user is authenticated, the operating system manages their permissions and access rights to files, directories, and other system resources. This involves determining who can read, write, execute, or modify data, ensuring that users and processes have access only to the information they are authorized to use.
- **Encryption:** Many operating systems provide built-in support for data encryption to protect sensitive information. Encryption converts data into a coded format that can only be accessed by those with the appropriate decryption key, making it much harder for attackers to read the data even if they gain access to it.
- **Process Isolation and Sandboxing:** The OS isolates running processes from one another to prevent one compromised process from affecting others or accessing their data. Sandboxing techniques further restrict applications by running them in a controlled environment with limited access to the system's resources.
- **Audit and Logging:** Operating systems keep logs of system activity, including user logins, file accesses, and system changes. These logs are crucial for detecting unauthorized access attempts, identifying security breaches, and performing post-incident analysis.
- **Patch Management and Updates:** Operating systems are responsible for updating their components and applying security patches regularly. These updates fix known vulnerabilities and weaknesses in the system, helping to protect against new threats and exploits.
- **Intrusion Detection and Prevention:** Some operating systems include built-in tools to detect and prevent suspicious activities, such as unusual network traffic or unauthorized file changes. These tools can alert administrators to potential security breaches and take automated actions to mitigate the threat.
- **Firewalls and Network Security:** Many operating systems come with integrated firewalls that control incoming and outgoing network traffic based on predefined security rules. This helps protect the system from unauthorized access, malware, and other network-based attacks.
- **Role-Based Access Control (RBAC):** Operating systems often use RBAC to manage user permissions by assigning roles to users based on their job functions. Each role has a defined set of permissions, making it easier to control who has access to specific resources.

5.2 Distributed Systems

Distributed computation requires communication between the various computers which is usually achieved through networks.

5.3 A Few Architectures

In the **Client-Server** architecture we have a client who requests a resource from the server, which carries out the request/command and sends the response back to the client.

In the **Middleware** architecture a third party/layer handles the details of communication and transmission. This is usually provided as a user space library which can be used from applications.

5.4 Messages

In the **Distributed Message Passing** architecture messages can be exchanged in the network. The **send(machine, process, message)** message specifies where the message should go while the **receive(machine, process, message)** tells the machine to listen for messages from specific nodes/processes.

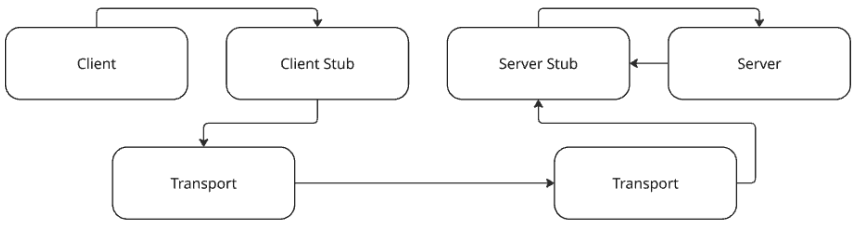
It's important to establish whether the communication is reliable or not, that is whether it's guaranteed that the message will arrive or not.

Whenever a message is sent we might want to wait until we know that the message arrived and in the meantime the process will go in a **blocking** state. If, on the other hand, we want to send the message and proceed with something else in our code the process we will resort to a **non-blocking** approach which is more efficient from a parallel processing point of view.

5.5 RPCs

RPC (Remote Procedure Calls) is a mechanism for executing a "procedure" on another machine. This normally means that the process starting a procedure needs the result and therefore it's a blocking process.

When a program makes an RPC it calls a **local stub** procedure. This stub is responsible for packaging the call information (such as function parameters) into a format that can be sent over the network to the remote system. The remote machine's OS then receives the request and the **server stub** unpacks it and calls the appropriate procedure on that machine. Once the procedure has finished executing, the result is sent back to the calling machine, where the local stub unpacks the response and presents it to the program as if it were a local result.



In synchronous RPC the calling program waits until the result is delivered while asynchronous RPC uses non-blocking communication.

5.6 Clusters

Clustering is the practice of linking multiple computers (often called nodes) to work together as a cohesive unit. A cluster acts as a single system from the user's perspective but it's made up of several inter-connected machines that share the workload.

The most desirable properties when building a cluster are:

- **Absolute scalability** → ability to significantly increase computational power or storage capacity by adding more machines to the cluster
- **Incremental scalability** → ability to expand the cluster without needing to redesign the system architecture
- **High availability** → if a node goes down it should not make the entire cluster fail
- **Superior price/performance** → it should be cost efficient

Beowulf is an example of a clustering system used to perform big computation. Processes are agnostic with respect to the device they are run on and they can also run on multiple nodes.

5.7 Questions

- The OS itself can run over multiple computers??

5.8 Notes